Technical Problem Statement: AI SLA-Escalation Bot for Discourse

Time allowed: 5 days

Format: A real, running bot on a free-trial Discourse instance the candidate sets up themselves + a short design doc (3–5 pages)

AI tools: Candidate's choice — OpenAI, Claude API, LangChain/LlamaIndex, or any RAG stack they're comfortable with

Knowledge source provided: 2 mock Neutrinos documentation PDFs + mock Community seed content (attached) to import into the trial Discourse instance

Deadline: Assignment starts today, Wednesday, 15 July 2026. Candidate has 5 days to complete and **must submit by Sunday, 19 July 2026**. Live presentation round is scheduled for **Monday, 20 July 2026**.

---

1. Problem Statement

> New members post questions in Neutrinos Community (built on Discourse). If a question isn't answered by a human within a defined SLA, it should be automatically answered by an AI bot — grounded in Neutrinos Documentation and prior resolved Community content — so no question is ever left hanging, without replacing genuine human expert engagement.

This is deliberately scoped as a **real build**, not a mockup:

1. Candidate signs up for a **free trial of Discourse** (self-hosted trial or Discourse-hosted trial — either is fine) and sets up a basic community.
2. Candidate seeds it with the provided mock Community content (attached: *Community_Seed_Content.md*) — a mix of already-solved threads and deliberately unanswered ones.
3. Candidate uses the provided mock **Neutrinos Documentation PDFs** (attached: 2 PDFs covering Workflow Builder and API/Integrations) as the core knowledge base. Note: there is deliberately **no** "Getting Started" documentation provided — Getting Started questions in the seeded community must be answered by the bot purely from prior resolved Community threads (or the bot must recognize it has no strong source and respond accordingly). This is an intentional gap to test retrieval honesty, not an oversight.
4. Candidate builds an AI bot that monitors new topics/posts via Discourse's webhook or API polling, tracks whether a human has replied within a defined SLA window, and — if the SLA is breached with no adequate human answer — triggers a RAG pipeline that retrieves relevant chunks from the Documentation PDFs **and** prior resolved Community threads, generates an answer, and posts it back into the Discourse thread as a reply clearly labeled as AI-generated.

2. Attached Materials (provided to candidate)

File	Purpose
Neutrinos_Workflow_Builder_Reference.pdf	Mock documentation — node types, error handling, versioning, troubleshooting
Neutrinos_API_Integration_Guide.pdf	Mock documentation — auth, connectors, rate limits, common integration errors
Community_Seed_Content.md	10 mock forum topics to post into the trial Discourse instance — 3 already solved by a human

These are intentionally realistic but fictional — they mirror the kind of content a real Neutrinos knowledge base would have (environment-scoped credentials, node types, rate limits, etc.) so the retrieval and answer-quality evaluation is meaningful, not just "does it find the right PDF page."

Deliberate gap: there is no "Getting Started" PDF in this set. The seeded Getting Started category topics have no direct documentation backing — only whatever a prior human reply happened to cover. This forces the bot to either lean on Community-only retrieval or correctly recognize low confidence and decline/escalate, rather than confidently answering from a source that doesn't exist.

3. Required Components

A. Discourse Setup

- Trial instance live and reachable
- At least the 4 categories from the seed content created (Getting Started, Workflow Builder, API & Integrations, Bugs & Troubleshooting)
- Seed topics posted, including the mix of solved/unsolved

B. SLA Monitoring

- A mechanism (webhook listener or polling script) that tracks each new topic's time-since-created and whether it has a qualifying human reply
- Candidate should define what counts as "answered" — e.g., a reply exists AND is marked as the topic's accepted solution vs. just any reply at all. **This distinction is one of the more important design decisions in this exercise** — topic 3 in the seed content is a deliberate edge case where a reply exists but doesn't actually resolve the question.

C. RAG Pipeline

- Ingest and chunk the 2 provided PDFs
- Ingest prior **resolved** Community threads as an additional retrieval source (the bot's knowledge base grows from the community itself, not just static docs)
- Retrieve relevant context and generate a grounded answer, mitigating hallucination — e.g., citing which doc/thread the answer came from, or declining to answer if retrieval confidence is low

D. Posting Back to Discourse

- Bot account posts the generated answer as a reply in the original thread via the Discourse API
- Reply should be clearly identifiable as AI-generated

- Bonus: bot response includes a "was this helpful?" prompt or a way to flag a human expert if the AI answer is insufficient

E. Guardrails

- What happens if the bot has low confidence / no good source? It should say so, not fabricate an answer — and ideally ping/mention a relevant human expert instead
- What stops the bot from answering a topic a human is actively mid-conversation on, just outside the SLA window? (e.g., don't fire if a human reply landed in the last N minutes even if not yet marked solved)

4. Deliverables

1. **Live demo** — screen recording or live walkthrough showing: a seeded unanswered topic → SLA breach → bot retrieves from docs/community → bot posts an answer in the actual Discourse thread
2. **Architecture diagram** — Discourse → webhook/polling → SLA checker → RAG pipeline → Discourse API post-back
3. **RAG design notes** — chunking strategy, retrieval method (keyword vs. embedding/vector search), which AI model used and why, how hallucination risk is mitigated
4. **SLA & "answered" definition** — written rationale for how the candidate decided a topic counts as human-answered vs. not
5. **Design doc** covering: how this would run continuously in production against live Neutrinos Community; how you'd measure whether the bot is actually helping; and the risk of the bot becoming the community's only voice and quietly killing human engagement

5. Evaluation Rubric

Dimension	What good looks like	Weight
End-to-end functionality	The bot actually runs against a real Discourse instance and posts a real reply on the test cases	25%
RAG quality & grounding	Answers are accurate, cite/reflect the source material, avoid hallucination	25%
SLA & "answered" logic	Thoughtful handling of edge cases (partial replies, in-progress human threads)	15%
Guardrails & judgment	Bot knows when NOT to answer, and doesn't undercut human experts	15%
Production & risk thinking	Realistic plan for running this live, and honest about the human-engagement risk	20%

Passing bar: The strongest candidates will treat "don't kill human engagement" as a first-class design constraint, not an afterthought — the goal is a bot that closes gaps the community isn't filling fast enough, not one that trains members to stop bothering to answer each other.

6. Presentation Round Structure — Monday, 20 July 2026

Segment	Duration
Live demo of the bot answering a seeded unanswered question	20 min

Segment	Duration
Panel tests an edge case live (e.g., new ambiguous question, or one with no doc coverage)	15 min
Discussion: production rollout plan + human-engagement risk mitigation	15 min
Candidate questions for panel	10 min